Promo tool integration

Introduction

This document describes API exposed by *Back Office*.

The key words `MUST`, `MUST NOT`, `REQUIRED`, `SHALL`, `SHALL NOT`, `SHOULD`, `SHOULD NOT`, `RECOMMENDED`, `MAY`, and `OPTIONAL` in this document are to be interpreted as described in RFC 2119.

GraphQL endpoint

GraphQL API is exposed under `/graphql` endpoint.

Introspection

GraphQl comes with fantastic introspection system which allows you to see the API.

The API schema is generated dynamically based on account permission so different account `MAY` see different schemas for different users.

There also GraphiQL web client available in the Back Office under `/backoffice/graphiql` to preview and test API in the browser.

Authorisation

The system use JWT authorisation with expiring keys. Mutation `login` generates new token.

Additionally, it checks time since last activity and requires re-authorisation after X hours (typically 4 hours) . It will return `SESSION_EXPIRED` error when the session expires.

Session expiry check can be skipped if the account has the whitelisted IPs configured and the calls come from those IPs.

```
curl 'http:/example.com/graphql' \
  -X POST \
  -H "Content-type: application/json" \
  -d '{"query":"mutation {login (email: \"account@example.com\" password: \"pass\") {token}}"}' -s \
  | jq ".data.login.token"
```

or

```
mutation {
    login(email: "my-account@example.com", password: "my-password") {
        token
    }
}
```

For the following requests you need to add the following header: `Authorization: Bearer <TOKEN>`

Campaigns

For exact API please check using introspection.

Methods

type	method	comment
`query`	`campaigns`	lists all campaigns
`query`	`campaignPlayers`	lists all campaigns
`query`	`campaignPrizes`	lists all campaigns
`mutation`	`addCampaign`	creates new campaign
`mutation`	`editCampaign`	edits the campaign
`mutation`	`deleteCampaign`	deletes the campaign

Campaign object

field		example	comment
`campaignId`	Auto-generated	`2fd45697-60af-4b98-8aae-fd9636c5cb1c`	Campaign Id
`name`	`OPTIONAL`	`My campaign`	Campaign name
`type`	`REQUIRED`	`freeBets`	Tool name
`config`	depends on the tool	`{"someData": 123}`	Campaign configuration
`start`	`OPTIONAL`	`1662624490000`	Start date of campaign (timestamp in milliseconds)
`end`	`OPTIONAL`	`1662624490000`	End date of campaign (timestamp in milliseconds)
`enabled`	`OPTIONAL` (default: `true`)	`true`	Indicates if campaign is enabled
`wallets`	`OPTIONAL`	`["wallet1", "wallet2"]`	Filters campaign to specified wallets
`operators`	`OPTIONAL`	`["operator1", "operator2"]`	Filters campaign to specified operators
`brands`	`OPTIONAL`	`["brand1", "brand2"]`	Filters campaign to specified brands
`providers`	`OPTIONAL`	`["provider1", "provider2"]`	Filters campaign to specified providers
`games`	`OPTIONAL`	`["game1", "game2"]`	Filters campaign to specified games
`playerIds`	`OPTIONAL`	`["2fd45697-60af-4b98-8aae-fd9636c5cb1c"]`	Filters campaign to specified player Ids
`nativeIds`	`OPTIONAL`	`["nativeId1", "nativeId2"]`	Filters campaign to specified native Ids (wallet player Ids)

Examples

Creating sample `freeBets` campaigns (it will return `campaignId` of newly created campaign.

```
mutation {
    addCampaign(data: {
        type: "freeBets",
        name: "test campaign",
        wallets: ["wallet1"],
        games: ["game1", "game2"],
        nativeIds: ["my-native-id"],
        config: {bets: 10, amount: 0.1, currency: "usd"}
    })
}
```

Deleting sample campaign:

```
mutation  {
    deleteCampaign(campaignId: "ab4ab94e-310d-4a3c-94a8-aa9a02b5675e")
}
```

Editing sample campaign (marking campaign as disabled):

```
mutation  {
    editCampaign(data: {enabled: false}, campaignId: "ab4ab94e-310d-4a3c-94a8-aa9a02b5675e")
}
```

Listing all `freeBets` campaigns:

```
query {
    campaigns (limit: 10, filter: {type: EQUAL, value: "freeBets", field: "type"}) {
        meta {
            hasNext hasPrev
        }
        items {
            campaignId type name
        }
    }
}
```

Tools

Free Bets

Transactions

Transactions under `freeBets` campaigns are marked with corresponding `campaignId` and `campaignType`. When `campaignType` equals `freeBets` it should not deduct this amount from player's balance.

Config

Param	Type	Required	Example	Comment
`bets`	`number`	`REQUIRED`	`10`	Number of free bets
`amount`	`number`	`REQUIRED`	`0.2`	Bet amount
`currency`	`string`	`OPTIONAL`	`eur`	Bet amount currency (if not specified it will use base currency (typically `eur`)

If player's currency is different from config `currency` it will be automatically converted to player's currency respecting the conversion ratio.

Available bets and games

You can download available bet configuration for given parameters

```
query {
    availableBets(currency: "sek", provider: "my-provider", operator: "my-operator", wallet: "my-wallet",
        game: "my-game", brand: "my-brand", jurisdiction: "my-jurisdiction") {
        bets
    }
}
```

You can fetch list of available games

```
query {
    availableGames(brand: "my-brand", operator: "my-operator", wallet: "my-wallet") {
        game
        provider
        title
        type
    }
}
```

Player feed

Requesting GET :

```
/feed/player/{campaignId}/{playerId}?currency=sek&provider=my-provider&game=my-game
```

or

```
/feed/player/{campaignId}/{wallet}/{nativeId}?currency=sek&provider=my-provider&game=my-game
```

will return current player state

```
{
  "total": 10,
  "used": 0,
  "left": 10,
```

```
  "amount": 5
}
```

In-game Jackpot

Feed

Requesting GET `/feed/campaign/{campaignId}?currency=sek` will return all tiers and their pools

```
{
  "mega": 100.233232,
  "mini": 5.0323
}
```

Prize Drop

Transactions

Transactions under `prizeDrop` campaigns give player a chance to award a fixed prize.

Config

Param	Type	Required	Example	Comment
`qualifyingBet`	`number`	`REQUIRED`	`10`	Bets greater or equal to this value will make the wager eligible for Prize Drop
`probability`	`number`	`REQUIRED`	`0.01`	Base fixed probability for triggering the Prize Drop award (the same for all bet sizes)
`boostedProbabilityStart`	`number`	`OPTIONAL`	`1662624490000`	Date from which the probability will start increasing linearly towards 1 (100%) at the End date
`prizes`	`IPrizeConfig[]`	`REQUIRED`		List of prizes for the campaign - each has the same probability of dropping

If player's currency is different from config `currency`, players qualifying bet will be automatically converted to player's currency respecting the conversion ratio.

Tournament

Transactions

Wining a positive amount during `tournament` campaigns places player in a leaderboard among other players participating in a campaign. After campaign is finished, players are awarded prices based on their `Win Ratio` scores. Win Ratio is defined as Total Round Win divided by "main" Game Bet. That means that if game has "Buy Bonus" feature, the Win Ratio will still take Base Game win as a ratio divisor. Note that in order for Win Ratio to work correctly, the game is required to define `coin` relationships between Base/Bonus game modes.

Config

Param	Type	Required	Example	Comment
`qualifyingBet`	`number`	`REQUIRED`	`10`	Bets greater or equal to this value will make the wager eligible for the Tournament leaderboard emplacement
`prizes`	`IPrizeConfig[]`	`REQUIRED`		List of prizes for the campaign, the order in configuration defines the leaderboard awards

If player's currency is different from config `currency`, players qualifying bet will be automatically converted to player's currency respecting the conversion ratio.

Common Types

IPrizeConfig

Param	Type	Required	Example	Comment
`type`	`cash/item/multiplier`	`REQUIRED`	`cash`	Prize type
`value`	`number/string`	`REQUIRED`	`1.12/"iPhone"`	Numerical value representing cash/bet multiplier to be paid to the player, or and item name, that represents what will player receive
`amount`	`number`	`REQUIRED`	`10`	The amount of prizes of a given type
`limit`	`number`	`OPTIONAL`	`100`	Cash limit affecting bet multiplier prizes

If player's currency is different from config `currency`, cash prizes will be automatically converted to player's currency respecting the conversion ratio. The algorithm used for rounding is designed in a way that makes prizes look meaningful for the players: the algorithm will take the amount of `significant digits` in the base currency cash value (eg. `1200,00` has two `significant digits`, `1200,10` has five `significant digits`) and round the converted value to the closest value that has (at most) the same amount of `significant digits`. On to of that, the algorithm might add one more "5" to the `significant digits` block if this value would be closer to the directly converted value. Eg. `100 * 4.251 -> 450`; `1500 * 0.33 -> 500`, `1500 * 0.77 -> 1150`